

From Algorithm to RTL

Radix-2² SDF FFT 설계 및 구현



(11조) 정민교 신상학 엄찬하
임재홍

목차



CHAPTER 01

개요

- 프로젝트 개요
- DFT / FFT 비교 및 필요성
- Radix 알고리즘 비교

CHAPTER 02

MATLAB을 이용한 FFT 분석

- float vs fixed point
- non_cbfp vs cbfp fixed
- SQNR 비교
- CBFP 분석

CHAPTER 03

SystemVerilog 설계

- HW Architecture
- Block Design
- Datapath
- Timing Diagram

CHAPTER 04

주요 모듈 소개

- Signal Scheduling
- CBFP1
- fac8_1 twiddle rotation

CHAPTER 05

시뮬레이션 결과 및 검증

- Simulation 분석
- MATLAB 출력 비교
- Random / Cosine 입력

CHAPTER 06

성능 분석 및 FPGA 구현

- Latency
- Area 및 Timing
- Vivado sim & Implementation

CHAPTER 07

트러블 슈팅

- Twiddle ROM
- Setup Violation
- Vivado_sim & implementation

CHAPTER 08

고찰 & 결론

- 고찰
- 결론



CHAPTER 01

개요

프로젝트 개요



발표 주제

Radix-2² SDF FFT 하드웨어 설계 및 검증

배경

Radix-2² SDF FFT

- 연산 효율성과 단순한 하드웨어 구조
- 자원 절감과 고속 처리
- 실시간 통신, 레이더, 영상 처리, AI 가속기 등 활용

목표

- (1) Setup Time Violation : MET 달성
- (2) 최적의 Area 만족
- (3) FPGA 보드에서 정상 작동

개발환경

언어 : SystemVerilog, Matlab

툴 : Vivado, Synopsys (Verdi, VCS)

DFT / FFT 비교 및 필요성



DFT

이산 푸리에 변환

시간 영역 -> 주파수 영역

연산량 : $O(N^2)$

N : 512 -> 약 262,144

FFT

고속 푸리에 변환

시간 영역 -> 주파수 영역

연산량 : $O(N \log_2 N)$

N : 512 -> 약 4,608

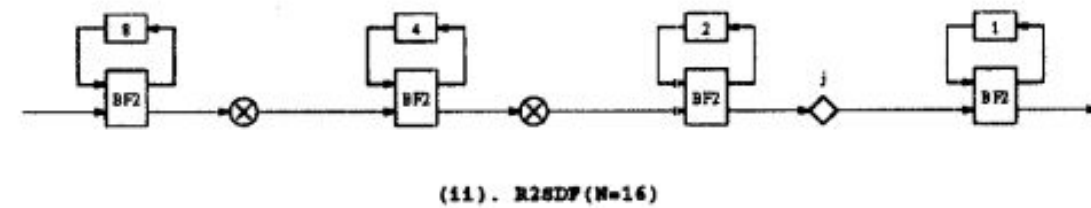
98% 감소

FFT : 계산 복잡도 감소



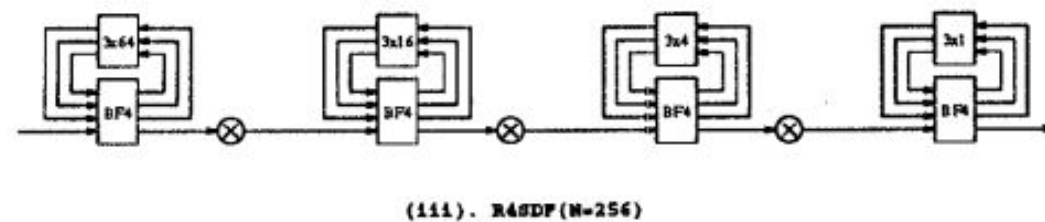
실시간 주파수 분석 가능

Radix 알고리즘 비교



Radix-2 Single-path Delay Feedback

- Multiplier : $2 (\log_4 N - 1)$
- Adder : $4 \log_4 N$
- Memory Size : $N - 1$



Radix-4 Single-path Delay Feedback

- Multiplier : $\log_4 N - 1$
- Adder : $8 \log_4 N$
- Memory Size : $N - 1$

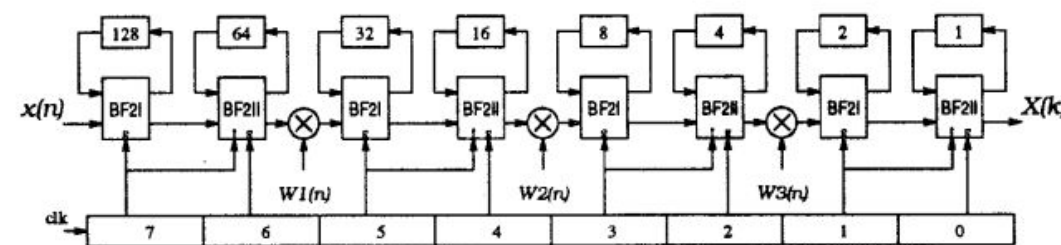


Figure 4: R2²SDF pipeline FFT architecture for $N = 256$

Radix-2² Single-path Delay Feedback

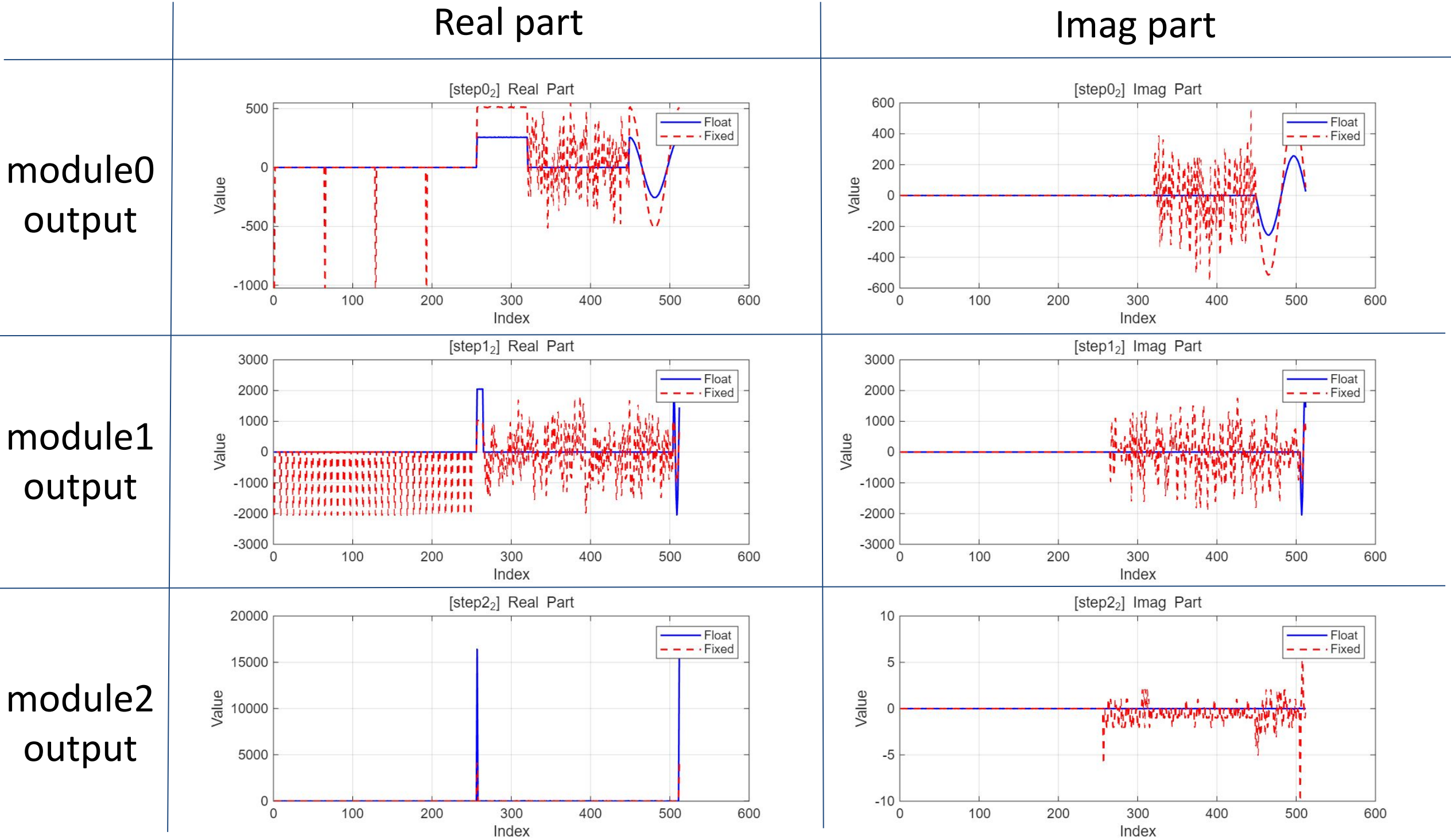
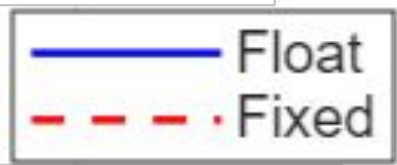
- Multiplier : $\log_4 N - 1$
- Adder : $4 \log_4 N$
- Memory Size : $N - 1$



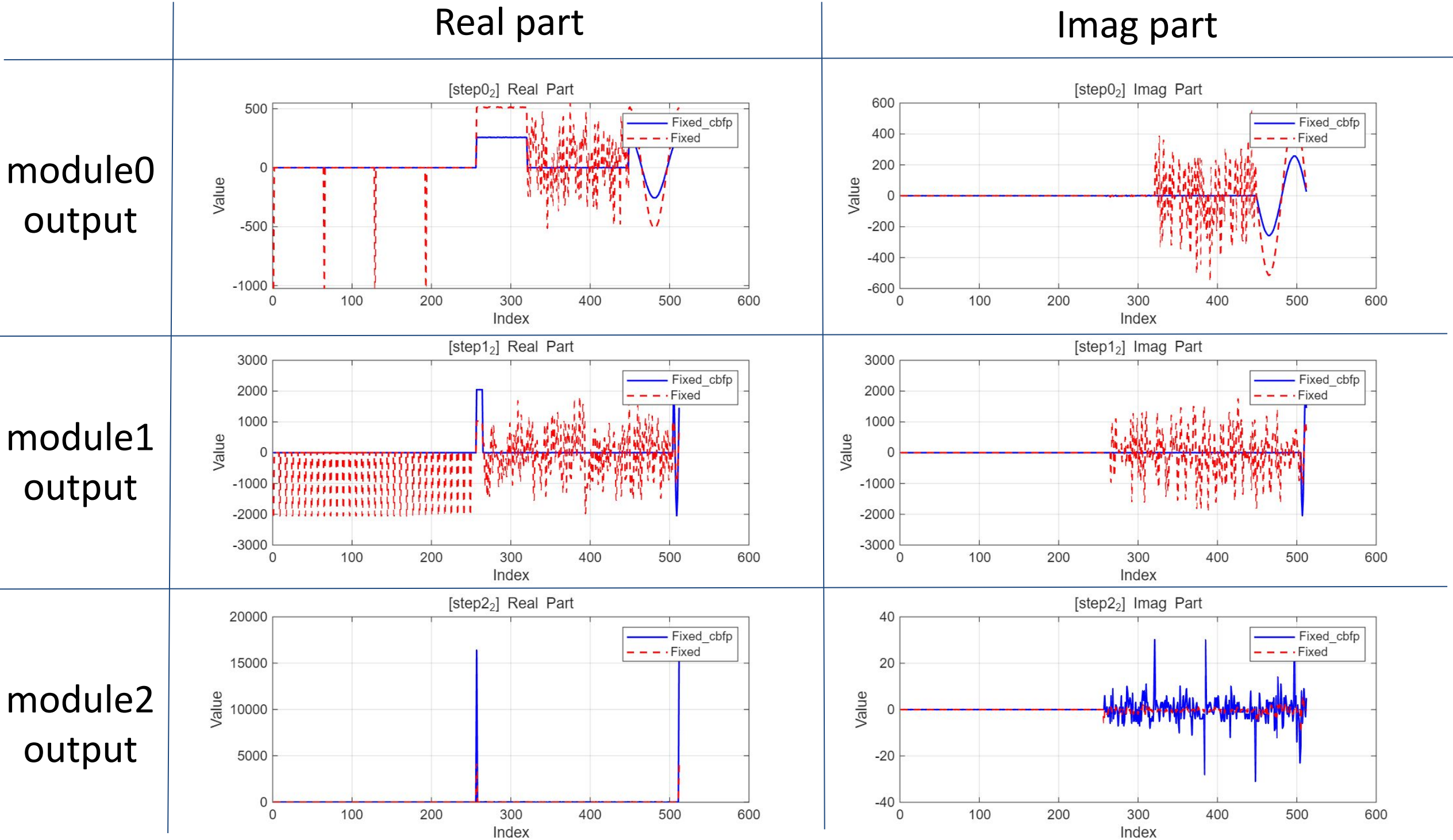
CHAPTER 02

MATLAB을 이용한 FFT 분석

● float vs fixed - MATLAB 그래프 비교

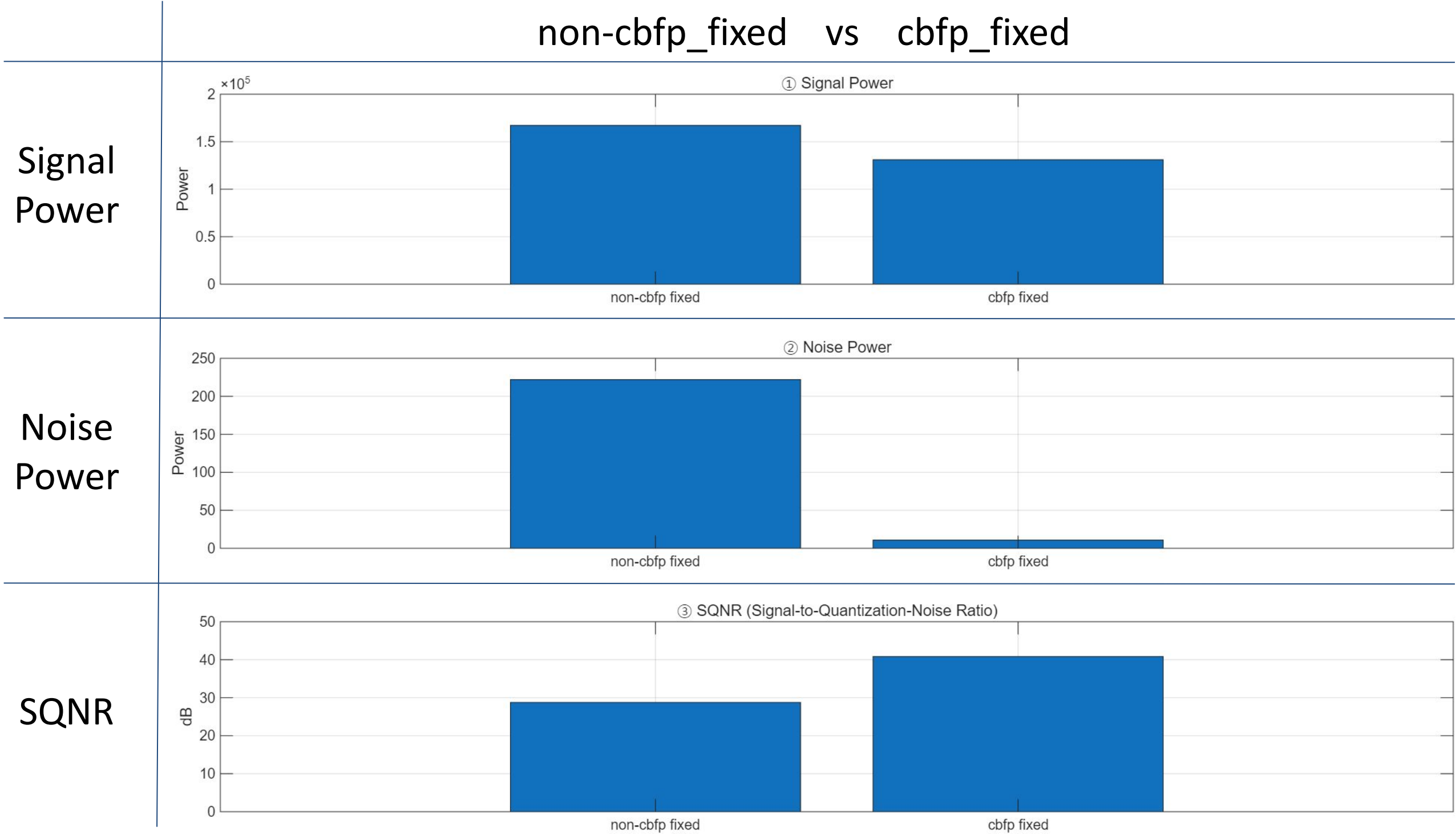


● fixed_cbfp vs fixed - MATLAB 그래프 비교



● SQNR - MATLAB 그래프 비교

- ① Signal Power
- ② Noise Power
- ③ SQNR



MATLAB - CBFP(Convergent Block Floationg-Point)



```
for nn=1:512
    bfly02(nn) = bfly02_tmp(nn)*twf_m0(nn); % <8.6> x <2.7> = <10.13>
    real_02 = round(real(bfly02(nn)/2^shift)); % <10.6>
    imag_02 = round(imag(bfly02(nn)/2^shift)); % <10.6>

    if(real_02 > 1023)
        real_02 = 1023;
    elseif(real_02 < -1024)
        real_02 = -1024;
    end

    if(imag_02 > 1023)
        imag_02 = 1023;
    elseif(imag_02 < -1024)
        imag_02 = -1024;
    end

    bfly02(nn) = real_02 + 1j * imag_02; % <5.6>
end
```


MATLAB - CBFP(Convergent Block Floationg-Point)

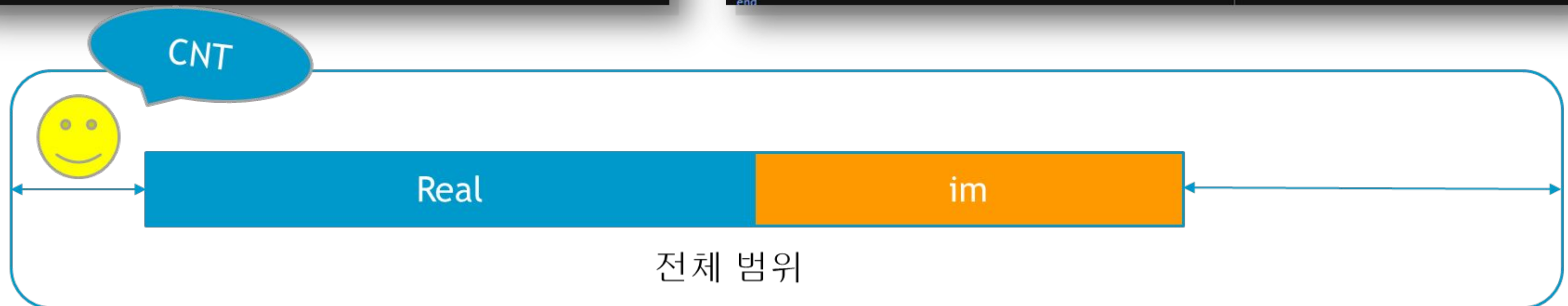


```
% CBFP(Convergent Block Floationg-Point) stage0
cnt1_re = zeros(1,8);
cnt1_im = zeros(1,8);
for ii=1:8
    for jj=1:64
        tmp1_re=mag_detect(real(pre_bfly02(64*(ii-1)+jj)),23);
        tmp1_im=mag_detect(imag(pre_bfly02(64*(ii-1)+jj)),23);
        % 블록 내에서 실수부/허수부 시프트 카운트의 최소값을 계속 갱신
        % (가장 큰 절댓값에 해당하는 가장 작은 시프트 카운트를 찾는 과정)
        temp1_re=min_detect(jj,tmp1_re,cnt1_re(ii));
        temp1_im=min_detect(jj,tmp1_im,cnt1_im(ii));
        cnt1_re(ii)=temp1_re;
        cnt1_im(ii)=temp1_im;
    end
end
```

```
% 각 블록에 대해 실수부와 허수부의 시프트 카운트 중 더 작은 값으로 통일
for ii=1:8
    if (cnt1_re(ii)<=cnt1_im(ii)) % 실수부 시프트 카운트가 더 작거나 같으면 그대로 유지
        cnt1_re(ii)=cnt1_re(ii);
    else
        cnt1_re(ii)=cnt1_im(ii); % 허수부 시프트 카운트가 더 작으면 실수부 시프트 카운트를 허수부로 맞춤 (더 보수적으로)
    end
end

for ii=1:8
    if (cnt1_im(ii)<=cnt1_re(ii)) % 허수부 시프트 카운트가 더 작거나 같으면 그대로 유지
        cnt1_im(ii)=cnt1_im(ii);
    else
        cnt1_im(ii)=cnt1_re(ii); % 실수부 시프트 카운트가 더 작으면 허수부 시프트 카운트를 실수부로 맞춤 (더 보수적으로)
    end
end

for ii=1:8
    for jj=1:64
        % 결정된 블록별 시프트 카운트를 전체 512개 데이터에 대해 복사
        index1_re(64*(ii-1)+jj)=cnt1_re(ii);
        index1_im(64*(ii-1)+jj)=cnt1_im(ii);
    end
end
```



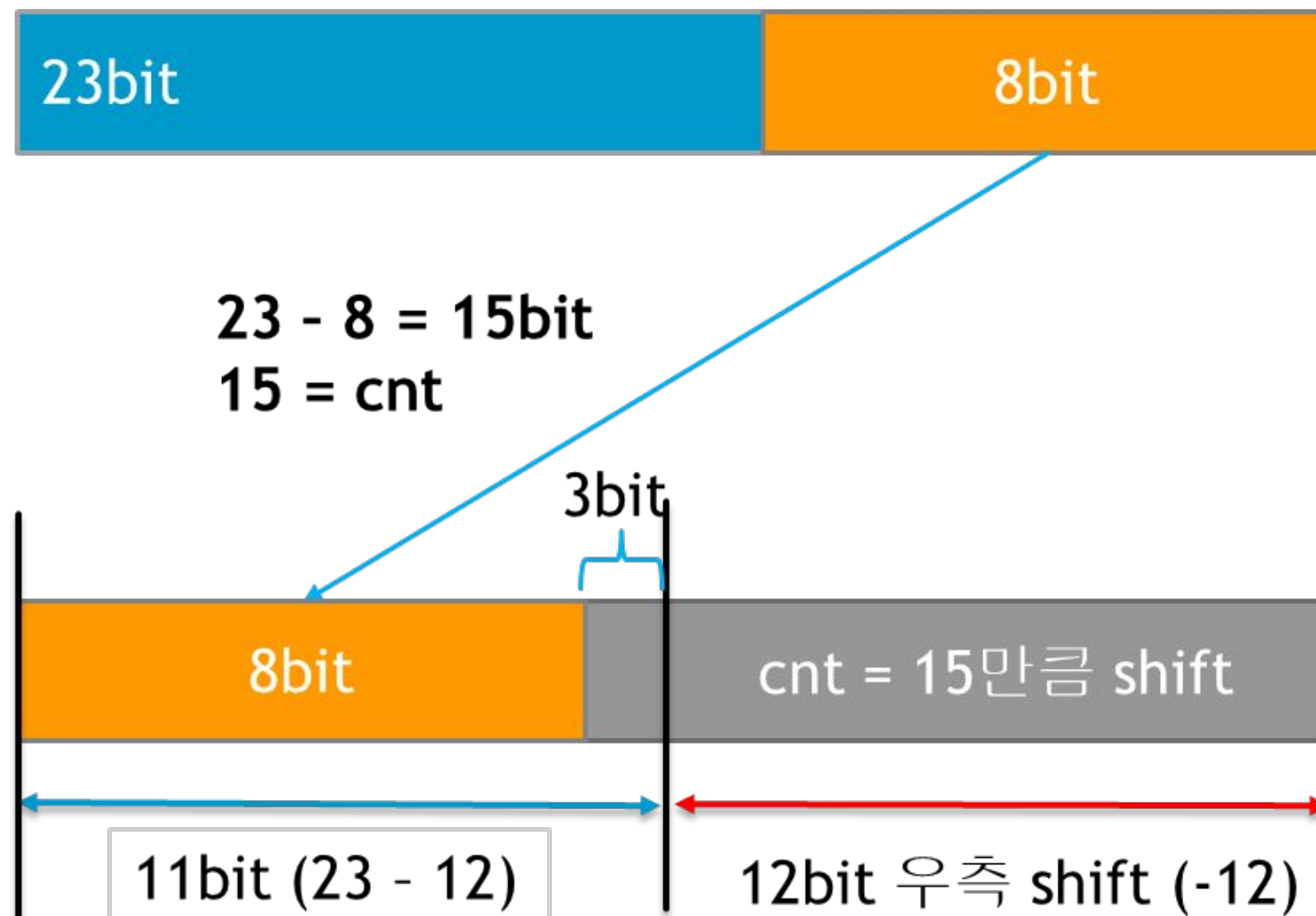
MATLAB - CBFP(Convergent Block Floationg-Point)



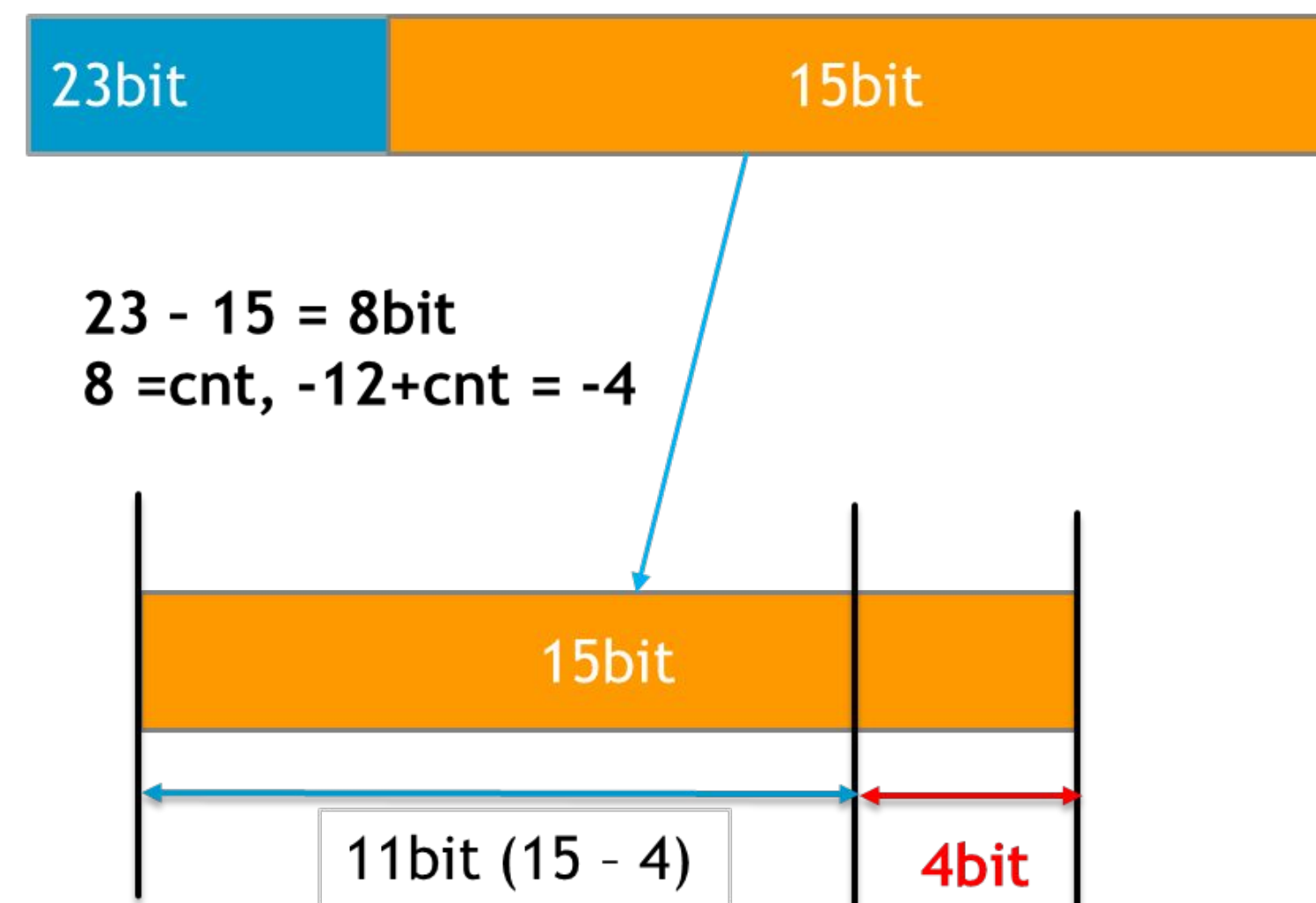
▶ bit shift

```
for ii=1:8
    for jj=1:64
        if (cnt1_re(ii)>12)
            re_bfly02(64*(ii-1)+jj)=bitshift(bitshift(real(pre_bfly02(64*(ii-1)+jj)),cnt1_re(ii), 'int32'),-12, 'int32');
        else
            re_bfly02(64*(ii-1)+jj)=bitshift(real(pre_bfly02(64*(ii-1)+jj)),(-12+cnt1_re(ii)), 'int32');
        end
    end
end
```

Cnt1_re>12



Cnt1_re<=12



MATLAB - CBFP(Convergent Block Floationg-Point)



23bit (전체 범위 <10.13>)

21bit -> 986,895

0__1111__0000__1111__0000__1111

14bit

0_1111__0000__1111__0

Round 2^7

11bit

011_1111_1111

Sat 11bit

011_1111_1111 => 1023 / $1023 * 2^{10} \approx 1,047,552$

$$\text{오차율} = \left(\frac{|1,047,552 - 986,895|}{986,895} \right) \times 100 \approx 6.15\%$$

23bit (전체 범위 <10.13>)

21bit -> 986,895

0__1111__0000__1111__0000__1111

21bit

0_1111__0000__1111__0000__1111

cnt = 2

11bit

011_1100_0011

>>> 12-cnt

011_1100_0011 => 963 / $963 * 2^{10} \approx 986,112$

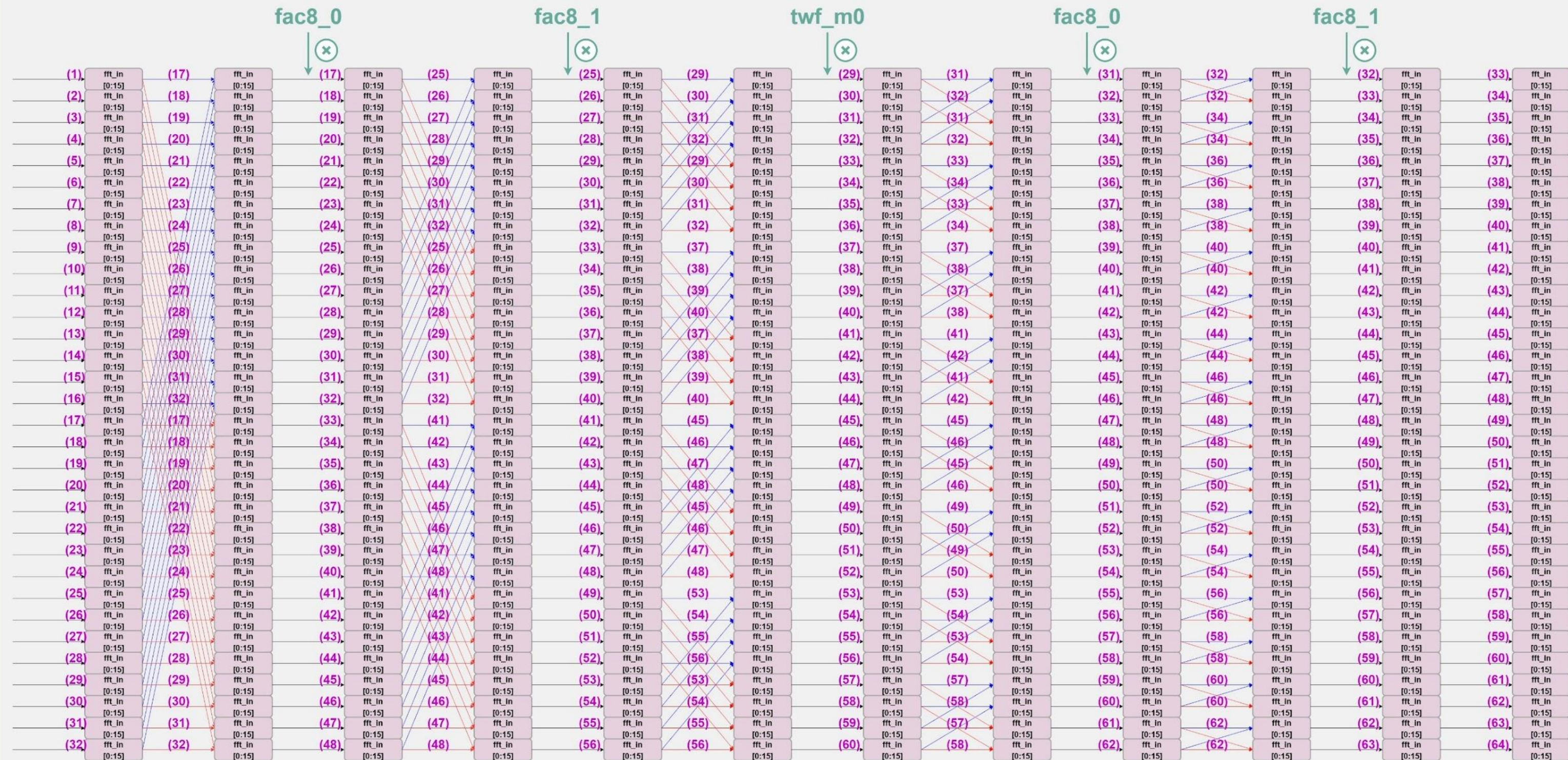
$$\text{오차율} = \left(\frac{|986,112 - 986,895|}{986,895} \right) \times 100 \approx 0.08\%$$



CHAPTER 03

System Verilog 설계

HW Architecture

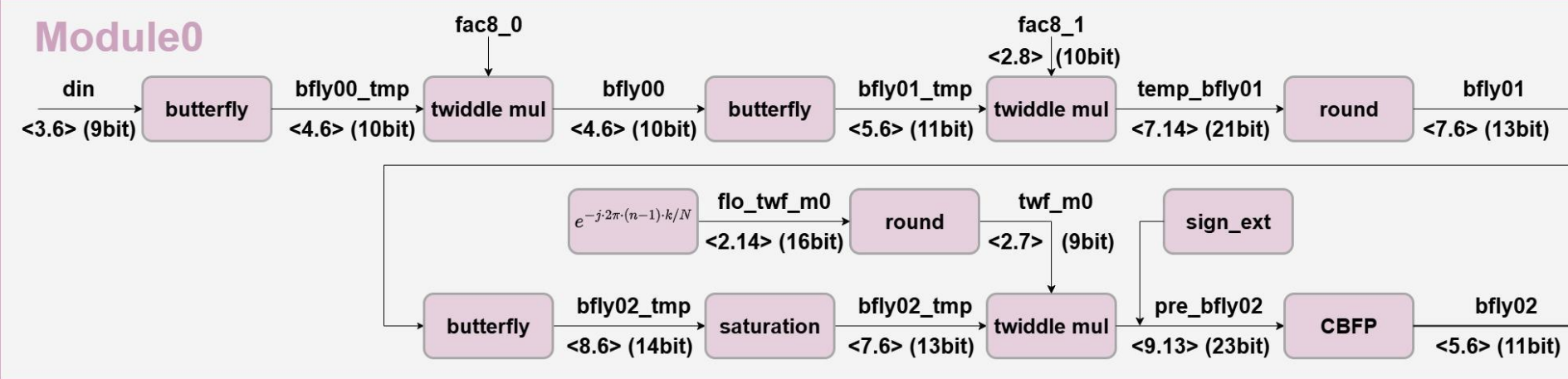


Block Diagram

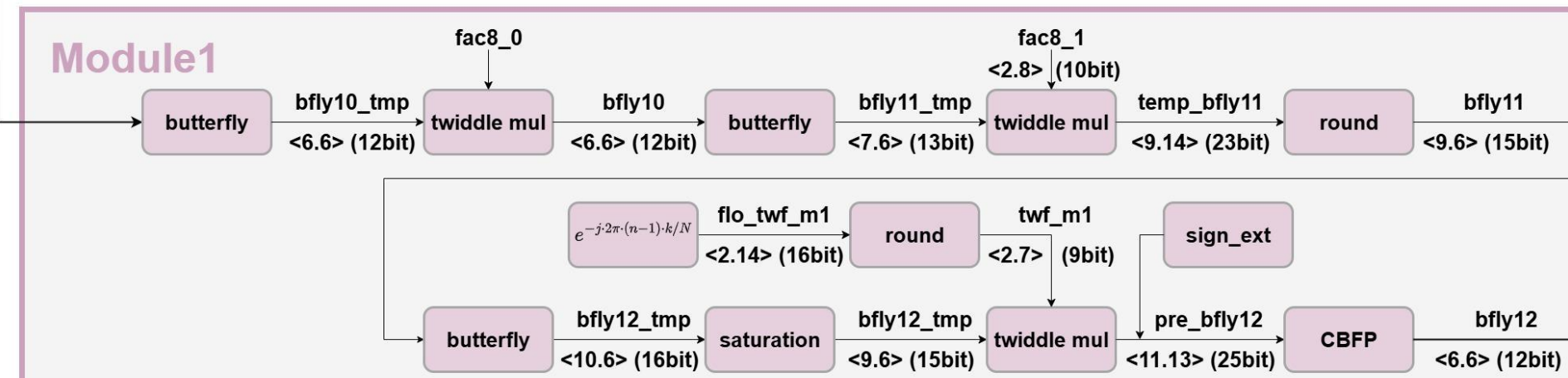


Fixed Point FFT

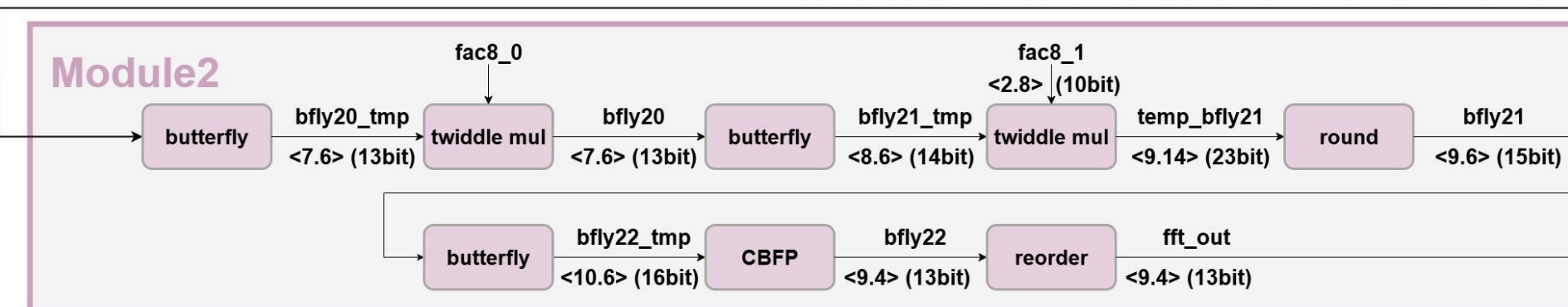
Module0



Module1



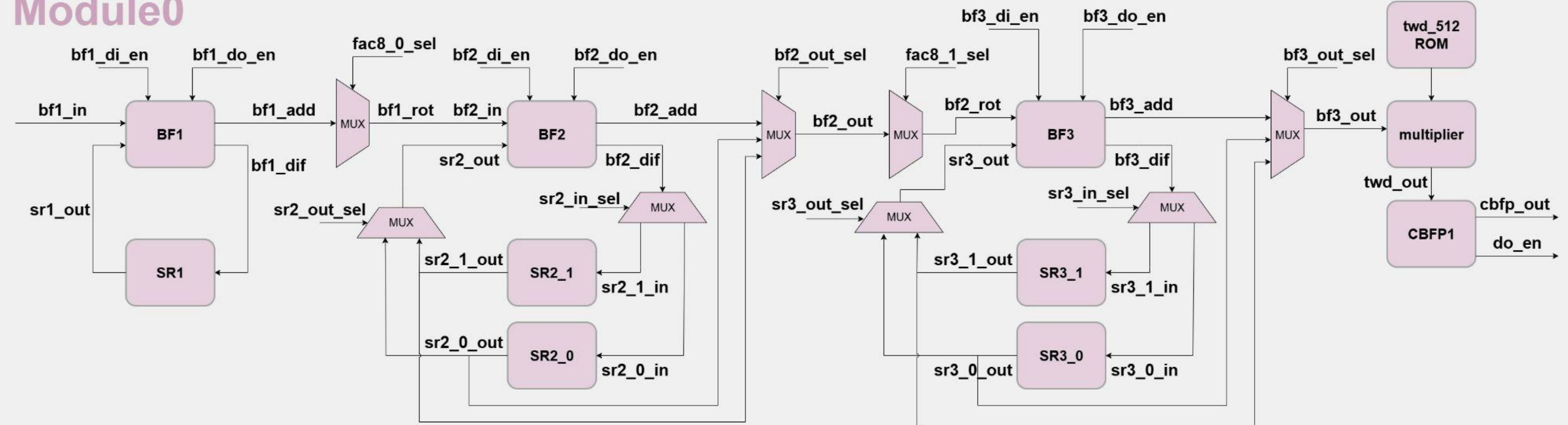
Module2



Datapath - Module0



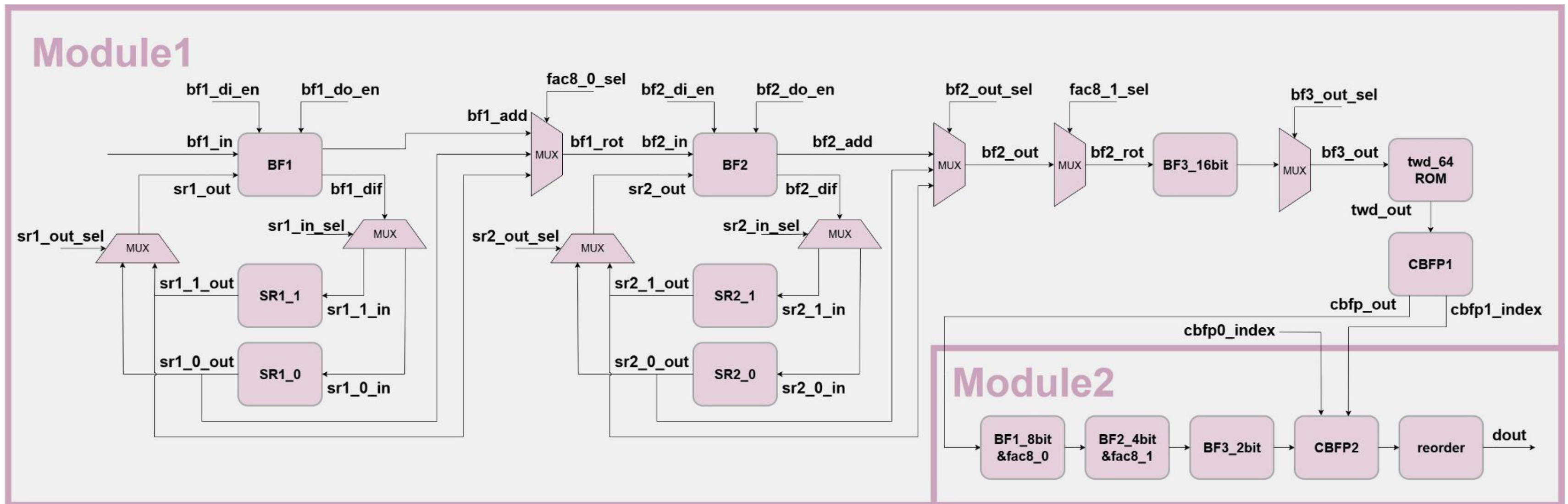
Module0



Timing Diagram



Datapath - Module1 & Module2





CHAPTER 04

주요 모듈 소개

주요 모듈 소개



1. 리소스 절약

MUX sel이나 enable 신호만으로 제어 가능하여 Register, LUT, Flip-Flop 등의 논리 자원 절약

2. 타이밍 분석 및 디버깅 용이

타이밍 다이어그램이 정적이고 waveform을 쉽게 검증 가능

3. 전력 소모 감소

실제 사용 시점에만 register가 클럭을 받아 불필요한 switching 차단

1) Signal hardcode scheduling

```
// 33~40 -> bf2_out_sel 1
if ((do_count >= 6'd33) && (do_count <= 6'd40)) bf2_out_sel = 2'd1;
// 49~56 -> bf2_out_sel 2
else if ((do_count >= 6'd49) && (do_count <= 6'd56)) bf2_out_sel = 2'd2;
else bf2_out_sel = 2'b00;

// fac8_1_sel 1
if ((do_count >= 6'd37) && (do_count <= 6'd40)) fac8_1_sel = 2'd1;
// fac8_1_sel 2
else if ((do_count >= 6'd45) && (do_count <= 6'd48)) fac8_1_sel = 2'd2;
// fac8_1_sel 3
else if ((do_count >= 6'd53) && (do_count <= 6'd56)) fac8_1_sel = 2'd3;
else fac8_1_sel = 1'b0;

// 29~60 -> bf3_out_en
if ((do_count >= 6'd29) && (do_count <= 6'd60)) bf3_out_en = 1'b1;
else bf3_out_en = 1'b0;

// 33~40 49~56 -> sr3_in_sel sr3_out_sel
if (((do_count >= 6'd33) && (do_count <= 6'd40))
    || ((do_count >= 6'd49) && (do_count <= 6'd56))) begin
    sr3_in_sel = 1'b1;
    sr3_out_sel = 1'b1;
end else begin
    sr3_in_sel = 1'b0;
    sr3_out_sel = 1'b0;
end
```



2) cbfp1 - shift 연산 index[0:7]

```
generate
  for (i = 0; i < 8; i = i + 1) begin : gen_block0_output
    // Block 0 시프트 연산 (인덱스 0~7)
    always_comb begin
      if (min_value_blk0 > 5'd13) begin
        temp_shift_re[i] = (di_re[i] <<< min_value_blk0) >>> 13;
        temp_shift_im[i] = (di_im[i] <<< min_value_blk0) >>> 13;
      end else if (min_value_blk0 == 5'd13) begin
        temp_shift_re[i] = di_re[i];
        temp_shift_im[i] = di_im[i];
      end else begin
        temp_shift_re[i] = di_re[i] >>> (5'd13 - min_value_blk0);
        temp_shift_im[i] = di_im[i] >>> (5'd13 - min_value_blk0);
      end
    end

    // Block 0 출력 할당
    assign do_re[i] = temp_shift_re[i][OWIDTH-1:0];
    assign do_im[i] = temp_shift_im[i][OWIDTH-1:0];

    // Block 0 인덱스 출력
    assign do_index[i] = min_value_blk0;
  end
end
```

index[8:15]

```
for (i = 0; i < 8; i = i + 1) begin : gen_block1_output
  // Block 1 시프트 연산 (인덱스 8~15)
  always_comb begin
    if (min_value_blk1 > 5'd13) begin
      temp_shift_re[i+8] = (di_re[i+8] <<< min_value_blk1) >>> 13;
      temp_shift_im[i+8] = (di_im[i+8] <<< min_value_blk1) >>> 13;
    end else if (min_value_blk1 == 5'd13) begin
      temp_shift_re[i+8] = di_re[i+8];
      temp_shift_im[i+8] = di_im[i+8];
    end else begin
      temp_shift_re[i+8] = di_re[i+8] >>> (5'd13 - min_value_blk1);
      temp_shift_im[i+8] = di_im[i+8] >>> (5'd13 - min_value_blk1);
    end
  end

  // Block 1 출력 할당
  assign do_re[i+8] = temp_shift_re[i+8][OWIDTH-1:0];
  assign do_im[i+8] = temp_shift_im[i+8][OWIDTH-1:0];

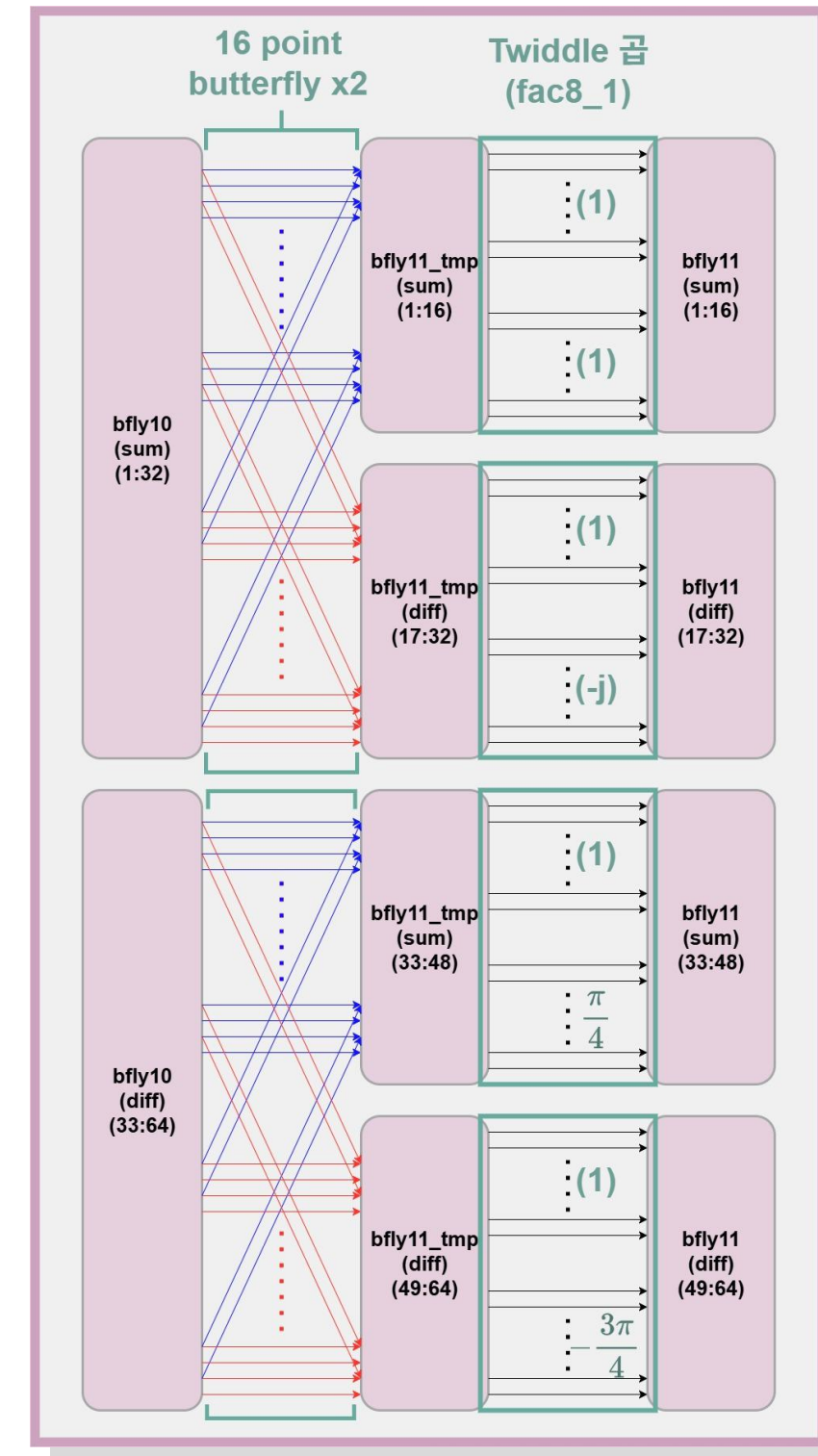
  // Block 1 인덱스 출력
  assign do_index[i+8] = min_value_blk1;
end
```


주요 모듈 소개



3) Module1 - fac8_1 rotation

```
always_comb begin
  for (int i = 0; i < 8; i++) begin
    case (fac8_1_sel)
      2'd0: begin
        bf2_rot_re[i] = -bf2_sign_re[i] * 9'sd256;
        bf2_rot_im[i] = -bf2_sign_im[i] * 9'sd256;
        bf2_rot_re[i+8] = -bf2_sign_re[i+8] * 9'sd256;
        bf2_rot_im[i+8] = -bf2_sign_im[i+8] * 9'sd256;
      end
      2'd1: begin
        bf2_rot_re[i] = -bf2_sign_re[i] * 9'sd256;
        bf2_rot_im[i] = -bf2_sign_im[i] * 9'sd256;
        bf2_rot_re[i+8] = -bf2_sign_im[i+8] * 9'sd256;
        bf2_rot_im[i+8] = bf2_sign_re[i+8] * 9'sd256;
      end
      2'd2: begin
        bf2_rot_re[i] = -bf2_sign_re[i] * 9'sd256;
        bf2_rot_im[i] = -bf2_sign_im[i] * 9'sd256;
        bf2_rot_re[i+8] = 9'sd181 * bf2_sign_re[i+8] + 9'sd181 * bf2_sign_im[i+8];
        bf2_rot_im[i+8] = 9'sd181 * bf2_sign_im[i+8] - 9'sd181 * bf2_sign_re[i+8];
      end
      2'd3: begin
        bf2_rot_re[i] = -bf2_sign_re[i] * 9'sd256;
        bf2_rot_im[i] = -bf2_sign_im[i] * 9'sd256;
        bf2_rot_re[i+8] = -9'sd181 * bf2_sign_re[i+8] + 9'sd181 * bf2_sign_im[i+8];
        bf2_rot_im[i+8] = -9'sd181 * bf2_sign_re[i+8] - 9'sd181 * bf2_sign_im[i+8];
      end
      default: begin
        bf2_rot_re[i] = -bf2_sign_re[i] * 9'sd256;
        bf2_rot_im[i] = -bf2_sign_im[i] * 9'sd256;
        bf2_rot_re[i+8] = -bf2_sign_re[i+8] * 9'sd256;
        bf2_rot_im[i+8] = -bf2_sign_im[i+8] * 9'sd256;
      end
    endcase
  end
end
```





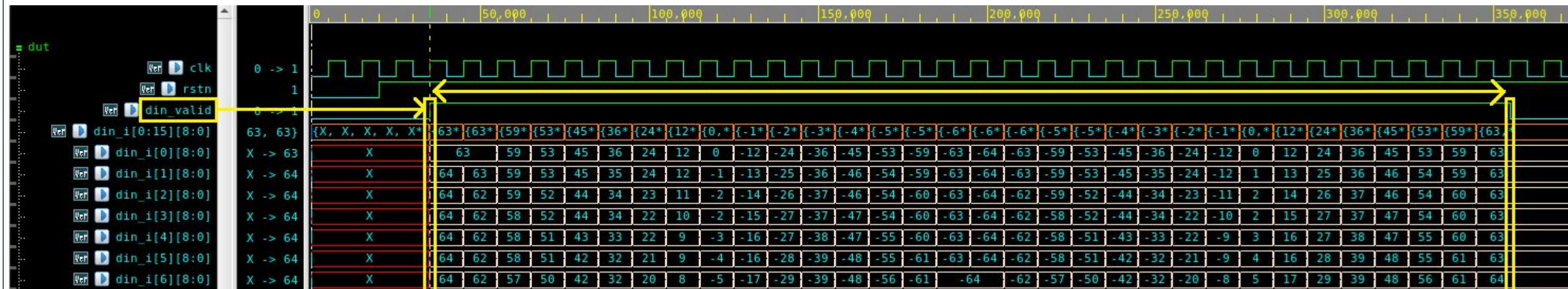
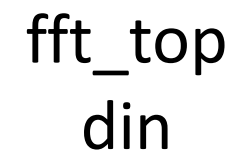
CHAPTER 05

시뮬레이션 결과 및 검증

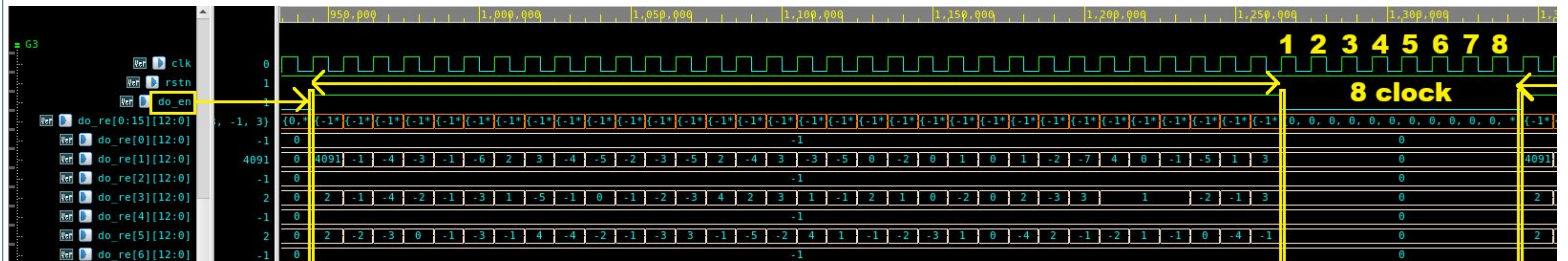
- VCS Result (COS)



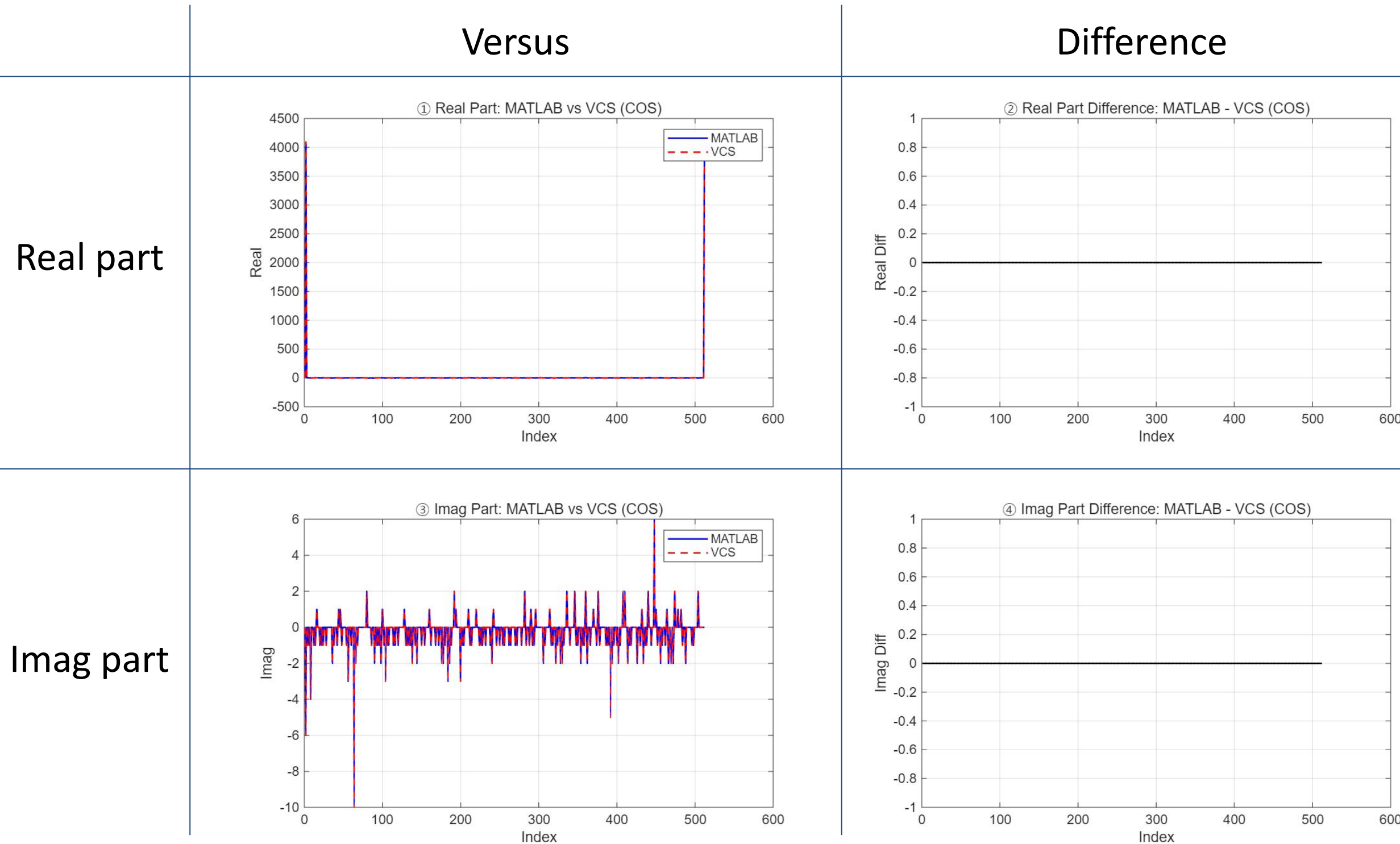
Waveform



fft_top
dout



● MATLAB vs VCS 결과 비교 (COS)

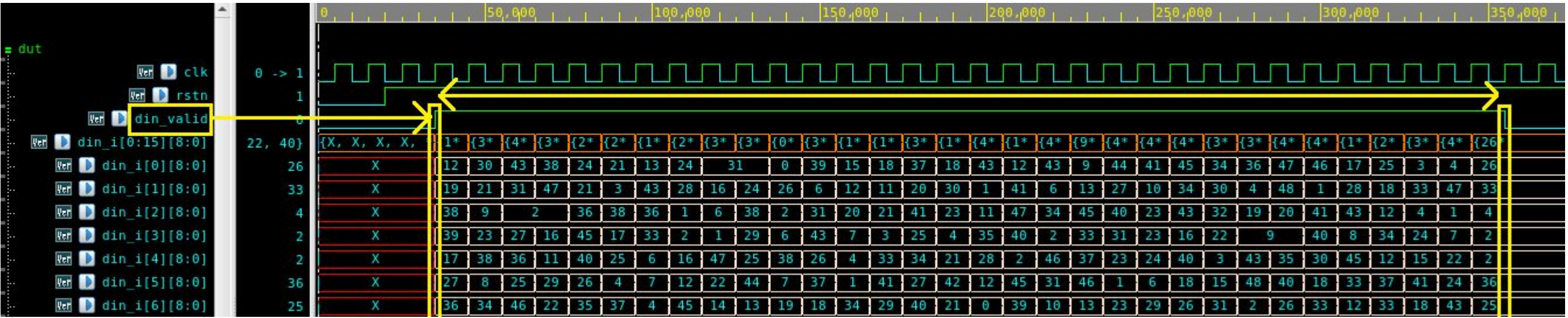


● VCS Result (RAN)

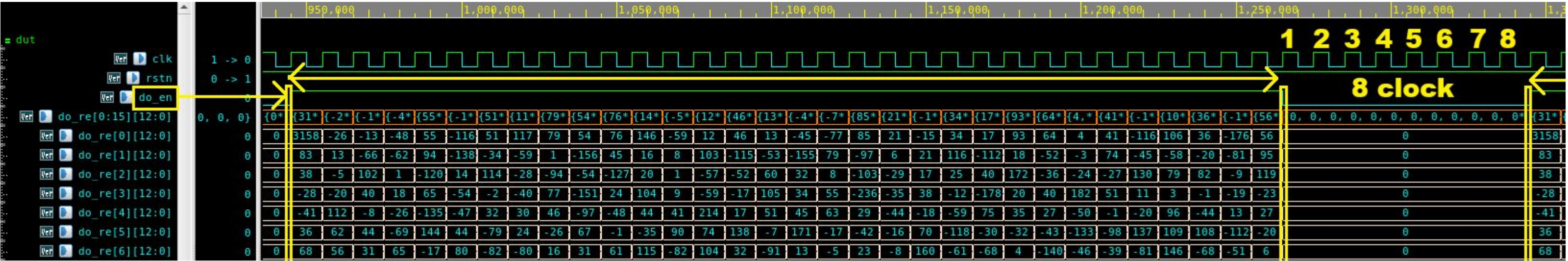


Waveform

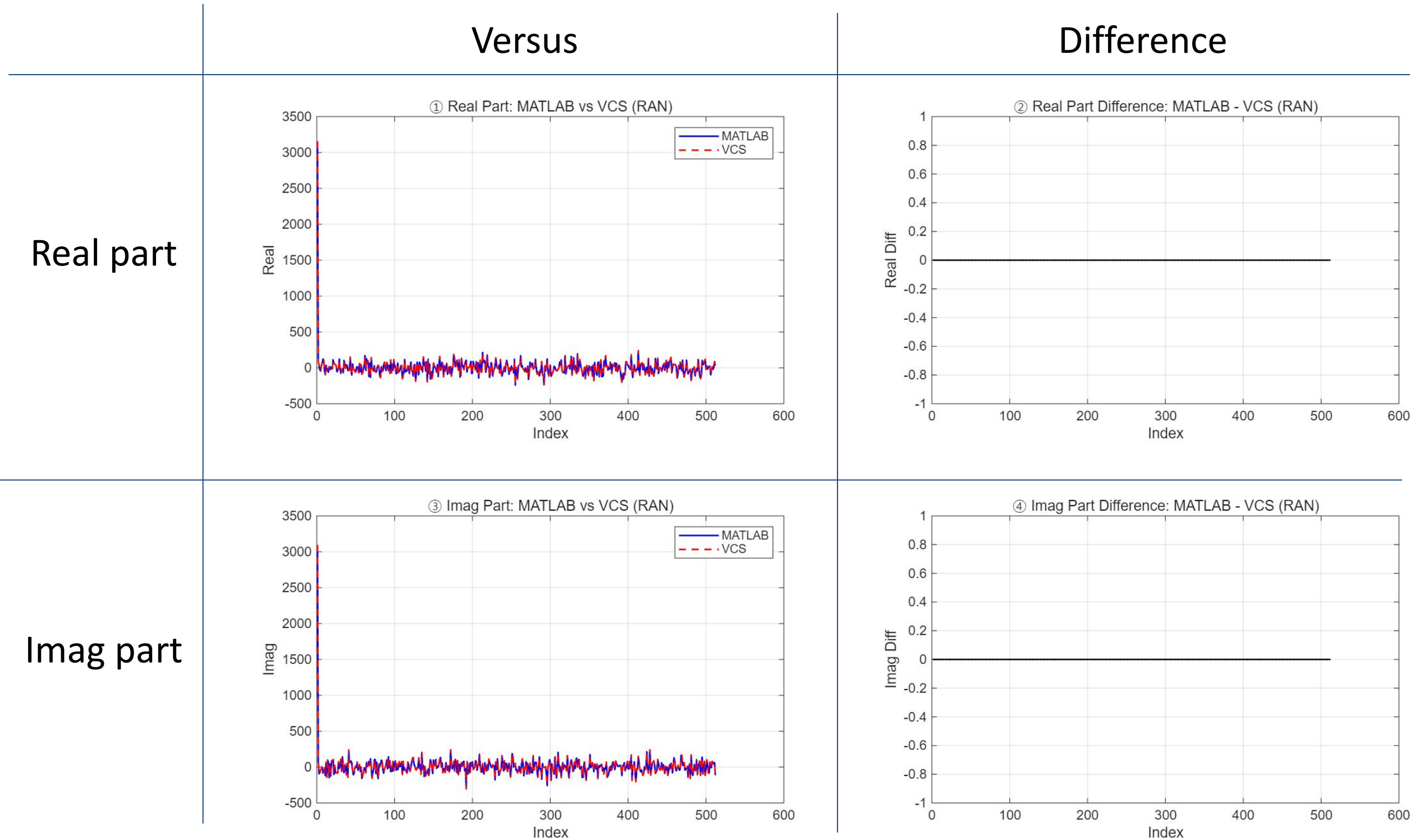
fft_top
din



fft_top
din



● MATLAB vs VCS 결과 비교 (RAN)





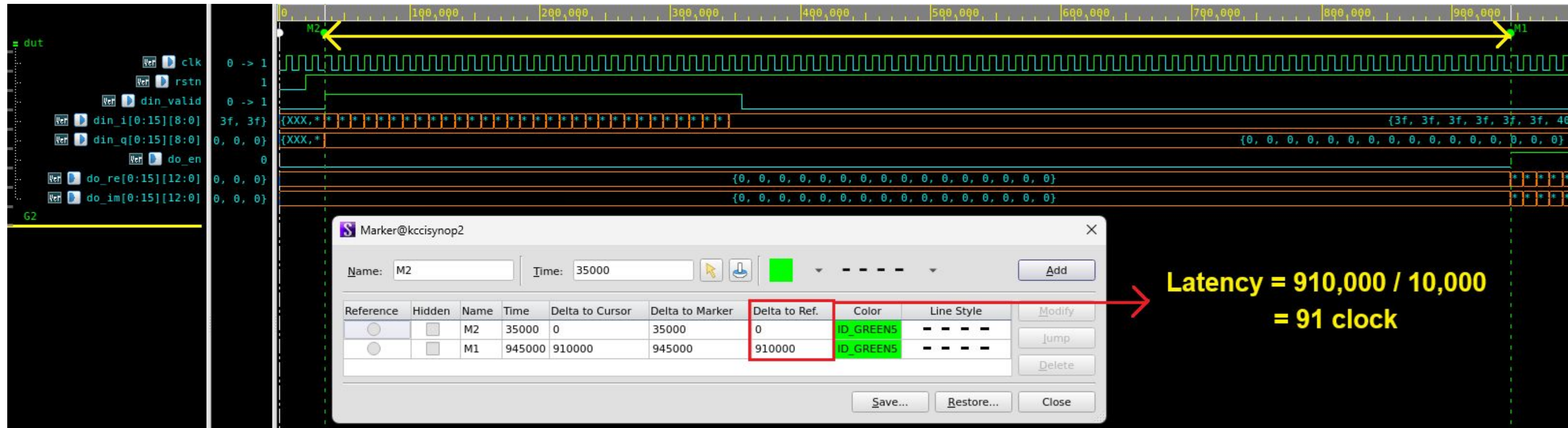
CHAPTER 06

성능 분석 및 FPGA 구현

• Latency



Waveform



입력부터 출력까지 걸리는 시간 = 91 clock

● Total cell area / slack (MET)



1 Synthesis 결과

Total cell area

Number of ports:	36802
Number of nets:	292894
Number of cells:	238992
Number of combinational cells:	179929
Number of sequential cells:	58882
Number of macros/black boxes:	0
Number of buf/inv:	33948
Number of references:	6
Combinational area:	88503.638117
Buf/Inv area:	8915.760020
Noncombinational area:	98945.238965
Macro/Black Box area:	0.000000
Net Interconnect area:	undefined (No wire load specified)
Total cell area:	187448.877082
Total area:	undefined

Value = 187,449

setup time slack (MET)

data arrival time		1309.50
clock fft_clk (rise edge)	1400.00	1400.00
clock network delay (ideal)	0.00	1400.00
clock uncertainty	-50.00	1350.00
MODUL0/sr3_1/shift_din_imag_reg_0__1__12_/CLK (SC7P5T_SDFFRQX2_A_CSC20L)	0.00	1350.00 r
library setup time	-40.49	1309.51
data required time		1309.51

data required time		1309.51
data arrival time		-1309.50

slack (MET)		0.01

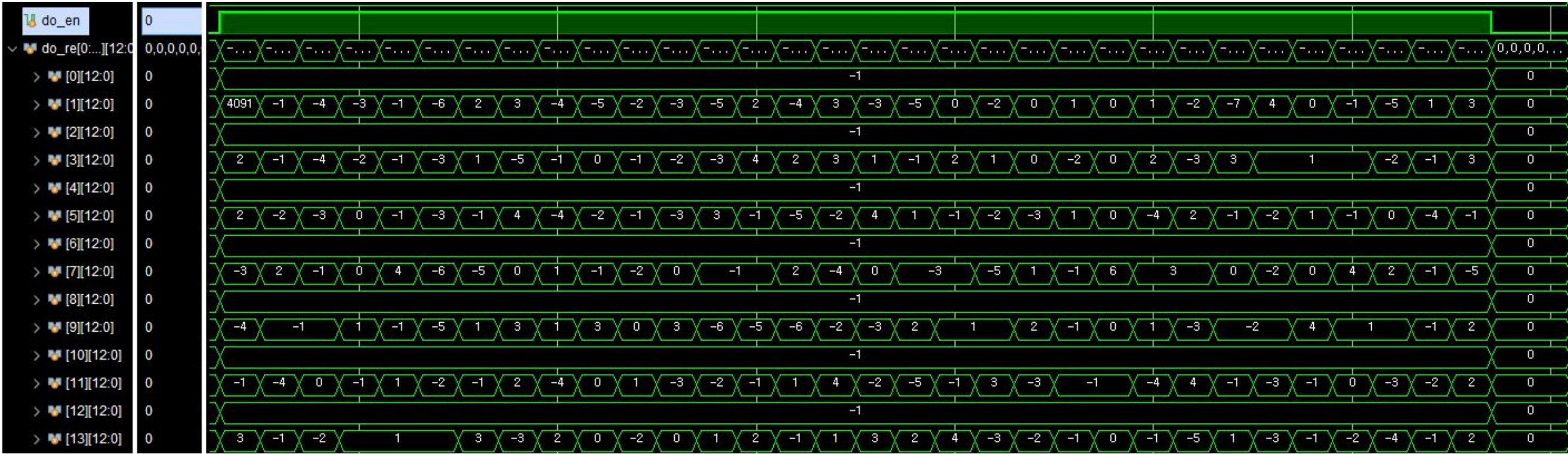
Value = 0.01

● Vivado_sim & implementation



Result

vivado_sim



implementation

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.003 ns	Worst Hold Slack (WHS): 0.019 ns	Worst Pulse Width Slack (WPWS): 2.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 94447	Total Number of Endpoints: 94431	Total Number of Endpoints: 47627

All user specified timing constraints are met.

● Vivado_sim & implementation

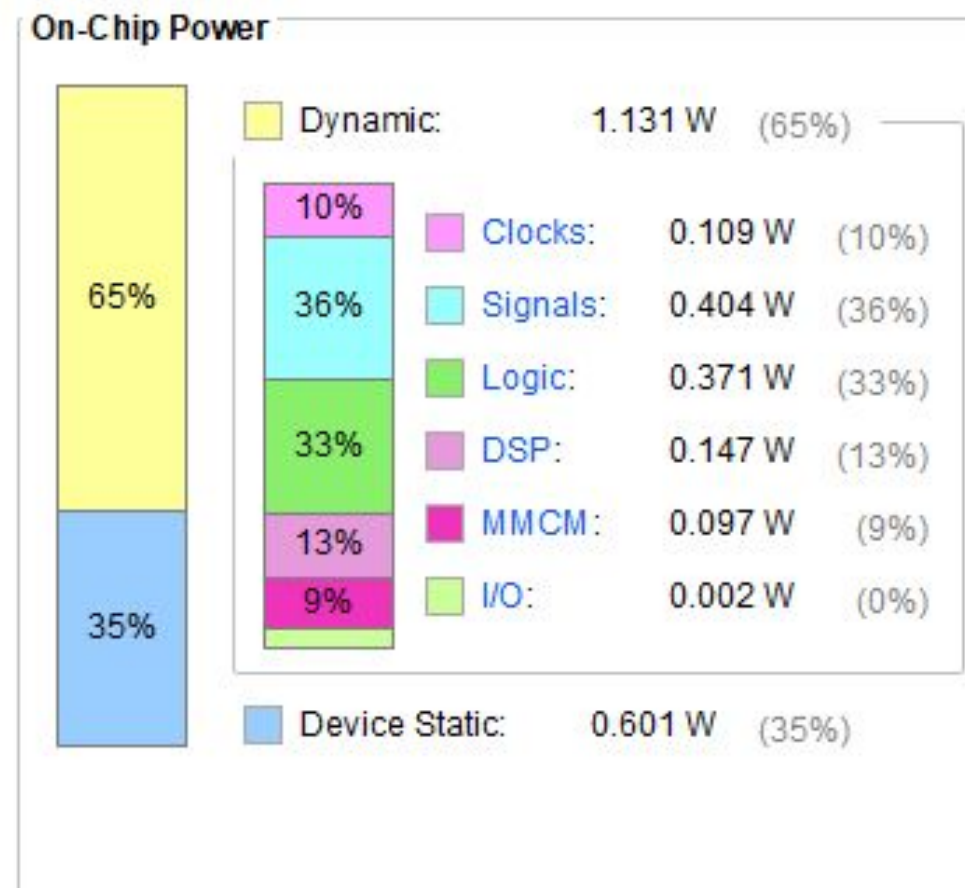


Power Consumption

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 1.732 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 27.4°C
Thermal Margin: 72.6°C (51.8 W)
Effective θ_{JA} : 1.4°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Medium

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity



높은 Static Power

: 높은 fanout, 장거리 배선

높은 Thermal Margin

: 온도 영향을 거의 받지 않음

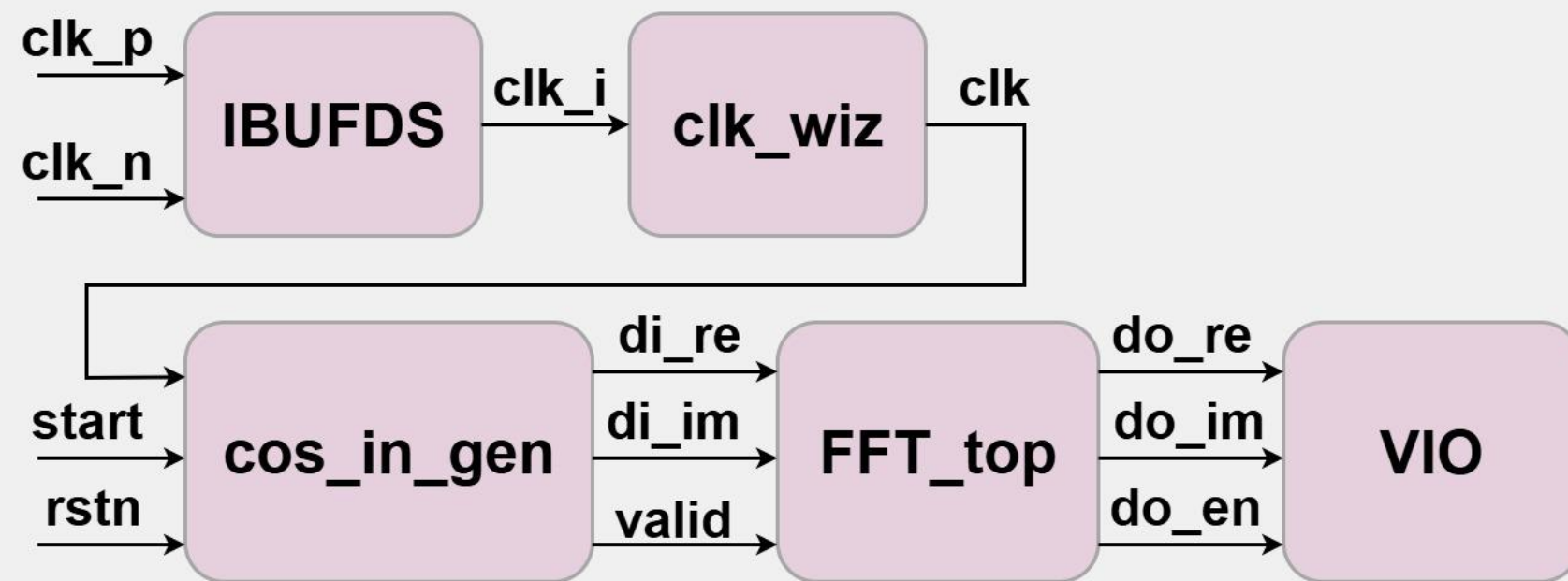
높은 Signal Power

: 배선에서 많은 toggle 발생

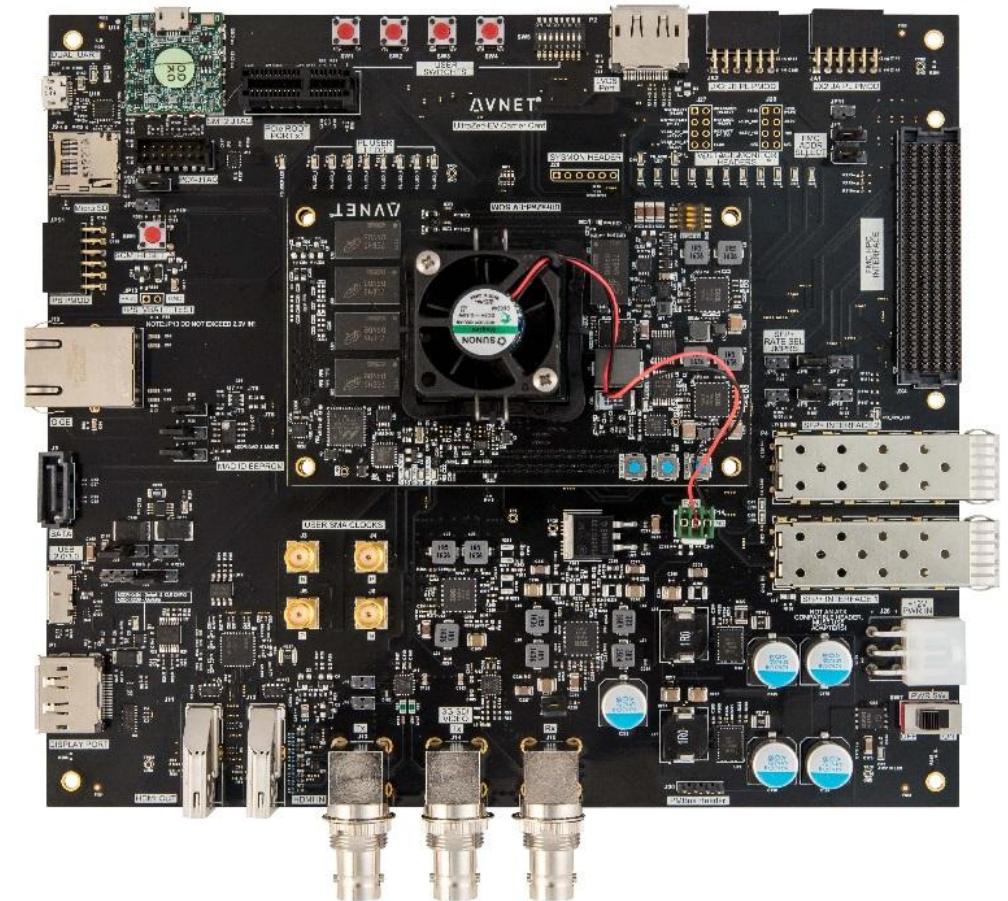
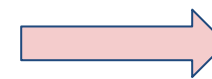
● FPGA 구현



FPGA



Generate Bitstream Complete



Ultrazed-7ev card



CHAPTER 07

트러블 슈팅

Trouble Shooting - Twiddle ROM (1)



1

초기 구상

한번에 값을 16개 출력하는 구조

```
input logic [$clog2(TW_TABLE_DEPTH)-1:0] addr [0:NUM_PARALLEL_PATHS-1],

output logic signed [WIDTH-1:0] tw_re [0:NUM_PARALLEL_PATHS-1],
output logic signed [WIDTH-1:0] tw_im [0:NUM_PARALLEL_PATHS-1]

genvar i;
generate
  for (i = 0; i < NUM_PARALLEL_PATHS; i++) begin : gen_twf
    logic signed [WIDTH-1:0] mx_re_i;
    logic signed [WIDTH-1:0] mx_im_i;
    logic signed [WIDTH-1:0] ff_re_i;
    logic signed [WIDTH-1:0] ff_im_i;

    assign mx_re_i = twf_m0_real_val[addr[i]];
    assign mx_im_i = twf_m0_imag_val[addr[i]];

    always @(posedge clk) begin
      ff_re_i <= mx_re_i;
      ff_im_i <= mx_im_i;
    end

    // 최종 출력 연결
    assign tw_re[i] = TW_FF ? ff_re_i : mx_re_i;
    assign tw_im[i] = TW_FF ? ff_im_i : mx_im_i;
  end
endgenerate
```

2

변경된 구조

top 모듈에서 for문으로 출력

```
input [$clog2(TW_TABLE_DEPTH)-1:0] addr,

output signed [WIDTH-1:0] tw_re,
output signed [WIDTH-1:0] tw_im

wire[8:0] twf_m0_real_val[0:TW_TABLE_DEPTH-1]; // Twiddle Table (Real)
wire[8:0] twf_m0_imag_val[0:TW_TABLE_DEPTH-1]; // Twiddle Table (Imag)
wire[8:0] mx_re; // Multiplexer output (Real)
wire[8:0] mx_im; // Multiplexer output (Imag)
reg [8:0] ff_re; // Register output (Real)
reg [8:0] ff_im; // Register output (Imag)

assign mx_re = twf_m0_real_val[addr];
assign mx_im = twf_m0_imag_val[addr];

always @(posedge clk) begin
  ff_re <= mx_re;
  ff_im <= mx_im;
end

assign tw_re = TW_FF ? ff_re : mx_re;
assign tw_im = TW_FF ? ff_im : mx_im;
```

Trouble Shooting - Twiddle ROM (2)



3

변경된 구조 Top modul
port save, 유지보수 극대화

```
// Twiddle Factor 주소 생성 (조합 로직)
always_comb begin
    for (int k = 0; k < NUM_PARALLEL_PATHS; k++) begin
        tw_addr[k] = (chunk_idx * NUM_PARALLEL_PATHS) + k;
    end
end

genvar t;
for (t=0 ;t<16;t++) begin
    twiddle_512 #(
        .WIDTH(TW_WIDTH),           // Twiddle Factor 데이터 비트 폭 (예: [WIDTH-1:0] = [8:0])
        .TW_TABLE_DEPTH(TW_TABLE_DEPTH), // 고유한 Twiddle Factor의 총 개수 (ROM 깊이)
        .TW_FF(1),                   // 출력 레지스터 사용 여부 (1: 사용, 0: 미사용)
        .NUM_PARALLEL_PATHS(NUM_PARALLEL_PATHS) // 추가된 파라미터 전달
    ) TWIDDLE_512(
        .clk(clk),
        .addr(tw_addr[t]),
        .tw_re(tw_re_out[t]),
        .tw_im(tw_im_out[t])
    );
end
```

2

변경된 구조
top 모듈에서 for문으로 출력

```
input [$clog2(TW_TABLE_DEPTH)-1:0] addr,
output signed [WIDTH-1:0] tw_re,
output signed [WIDTH-1:0] tw_im
```

```
wire[8:0] twf_m0_real_val[0:TW_TABLE_DEPTH-1]; // Twiddle Table (Real)
wire[8:0] twf_m0_imag_val[0:TW_TABLE_DEPTH-1]; // Twiddle Table (Imag)
wire[8:0] mx_re; // Multiplexer output (Real)
wire[8:0] mx_im; // Multiplexer output (Imag)
reg [8:0] ff_re; // Register output (Real)
reg [8:0] ff_im; // Register output (Imag)
```

```
assign mx_re = twf_m0_real_val[addr];
assign mx_im = twf_m0_imag_val[addr];
```

```
always @(posedge clk) begin
    ff_re <= mx_re;
    ff_im <= mx_im;
end
```

```
assign tw_re = TW_FF ? ff_re : mx_re;
assign tw_im = TW_FF ? ff_im : mx_im;
```


Trouble Shooting - Setup Violation



Setup time violation



Flip-Flop을 통한 PipeLine

Point	Incr	Path
clock cnt_clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
shift_din_reg_23__0_/CLK (SC7P5T_DFFRQX4_S_CSC20L)	0.00	0.00 r
shift_din_reg_23__0_/Q (SC7P5T_DFFRQX4_S_CSC20L)	47.04	47.04 f
shift_din_reg_24__0_/D (SC7P5T_DFFRQX1_AS_CSC20L)	0.00	47.04 f
data arrival time		47.04

clock cnt_clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
clock uncertainty	50.00	50.00
shift_din_reg_24__0_/CLK (SC7P5T_DFFRQX1_AS_CSC20L)	0.00	50.00 r
library hold time	13.32	63.32
data required time		63.32

data required time		63.32
data arrival time		-47.04

slack (VIOLATED)		-16.27

data arrival time		1309.50

clock fft_clk (rise edge)	1400.00	1400.00
clock network delay (ideal)	0.00	1400.00
clock uncertainty	-50.00	1350.00
MODUL0/sr3_1/shift_din_imag_reg_0__1__12_/CLK (SC7P5T_SDFFRQX2_A_CSC20L)	0.00	1350.00 r
library setup time	-40.49	1309.51
data required time		1309.51

data required time		1309.51
data arrival time		-1309.50

slack (MET)		0.01

Trouble Shooting -> Vivado_sim & implementation



2. REPORT DETAILS

DPIP-2#1 Warning

Input pipelining

DSP d_fft/MODUL0/bf2_rot_re1 input d_fft/MODUL0/bf2_rot_re1/A[29:0] is not pipelined. Pipelining DSP48 input will improve performance.

Related violations: <none>

DPIP-2#2 Warning

Input pipelining

DSP d_fft/MODUL0/bf2_rot_re1__0 input d_fft/MODUL0/bf2_rot_re1__0/A[29:0] is not pipelined. Pipelining DSP48 input will improve performance.

Related violations: <none>

DPIP-2#49 Warning

Input pipelining

DSP d_fft/MODUL0/cbfp1/C_MULTIPLIER_02/parallel_path[0].ac_term_reg_reg input d_fft/MODUL0/cbfp1/C_MULTIPLIER_02/

Related violations: <none>

```
logic signed [8:0] bf2_sign_re_reg[0:15], bf2_sign_im_reg[0:15];

always_ff @(posedge clk) begin
    for (int i = 0; i < 16; i++) begin
        bf2_sign_re_reg[i] <= bf2_sign_re[i];
        bf2_sign_im_reg[i] <= bf2_sign_im[i];
    end
end
```

implementation

Setup

Worst Negative Slack (WNS): -0.803 ns

Total Negative Slack (TNS): -392.198 ns

Number of Failing Endpoints: 1033

Total Number of Endpoints: 88217



Setup

Worst Negative Slack (WNS): 0.003 ns

Total Negative Slack (TNS): 0.000 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 94447

All user specified timing constraints are met.



CHAPTER 08

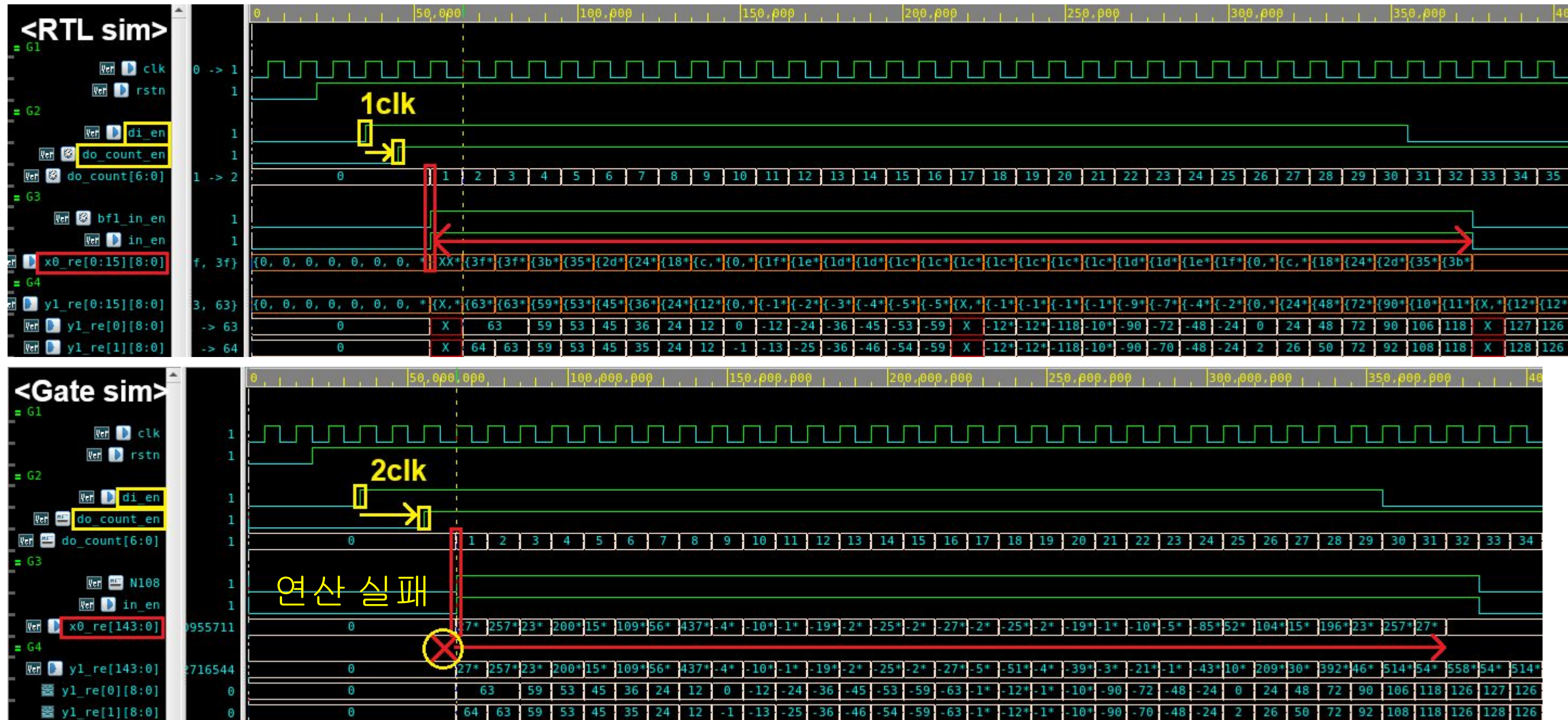
고찰 & 결론



고찰 - RTL sim vs Gate sim (Timing issue)

RTL 1clk vs Gate 2clk

- ▶ 입력 데이터(x0_re)는 이미 도착했지만, do_count_en이 준비되지 않아 연산 실패 발생
- ▶ 이 문제를 완전히 해결하지 못했으며, Gate 시뮬레이션의 현실적인 지연 고려 부족





본 설계에서는 타이밍 차트를 기반으로 카운터를 활용해 각 모듈의 동작을 정밀하게 제어하였다.

하지만 실제 Vivado에서 구현(implementation)을 진행한 결과, 일부 경로에서 슬랙이 발생하였고, 이를 해결하기 위해 FF 삽입 및 파이프라인 재구성이 필요했다.

그러나 카운터 기반 제어 방식은 클럭 단위로 동작이 촘촘하게 맞물려 있어 구조를 수정하는 데 상당한 제약이 있었다.

이러한 경험을 바탕으로, 향후에는 FSM 기반의 상태 제어 구조를 도입하여 각 연산 단계의 논리적 분리를 명확히 하고, 파이프라인 구성 및 유지보수의 유연성을 확보하는 것이 바람직할 것으로 판단된다.



THANK YOU!

Q&A